

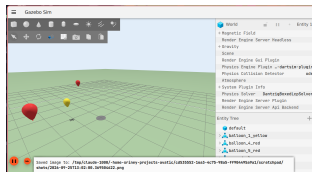
AVATIC

Autonomous Vision-Based Aerial Target Interception Challenge

INFINIUM '26 • Workshop

DRONE ARENA @ Infinium

github.com/Srindot/AVATIC



Today

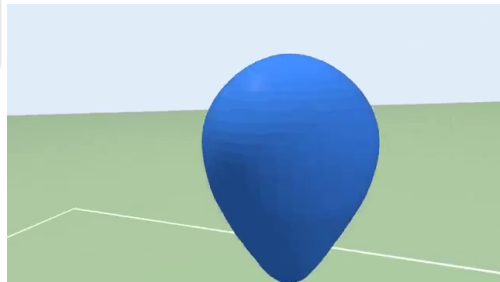
- 1 The objective
- 2 Game rules: balloons and points
- 3 Overview: the drone, the simulator, the two rounds
- 4 Rounds, deadlines and what we expect
- 5 The simulation round in depth
- 6 Solution ideas (a starting point, not a solution)
- 7 Sim2real: from the simulator to the real Pluto X
- 8 How teams are shortlisted
- 9 Deadlines, once more
- 10 Prizes
- 11 Questions

The objective

Your mission

Write a program that flies a **Pluto X** drone into balloons using **only its camera**.

- Your algorithm runs on **your laptop**.
- It receives the **video from the drone**, decides how to fly, and sends **flight commands** back.
- Pop the good balloons, avoid the red ones, in **25 seconds**.
- No GPS, no position, no map: just the camera and the drone's own readings (tilt, heading, height).



What your program sees, just before a pop

Game rules: balloons and points

	Balloon	Points	How many
●	green	+100	3
●	blue	+50	5
●	yellow	+25	6
●	red	−300	4

Best possible score: 700

- **18 balloons**, the more points the rarer; 30 cm wide, 0.8–2 m high, 1–3.5 m from the start
- more good balloons than anyone can pop: **choose well**
- a balloon **pops when the drone touches it**
- **25 s** from arming; later pops don't count
- red **costs points**: the score can go negative
- the drone starts facing east: balloons are often **behind** it
- **random layouts**: judging uses layouts nobody has seen

Overview: two rounds, one program

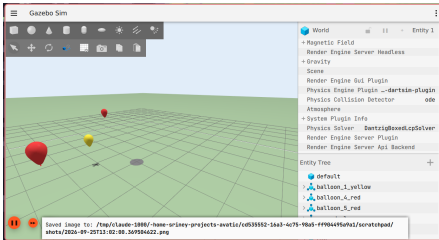
Round 1: Simulation

every team
your code flies the simulated Pluto X
in Gazebo, on hidden layouts

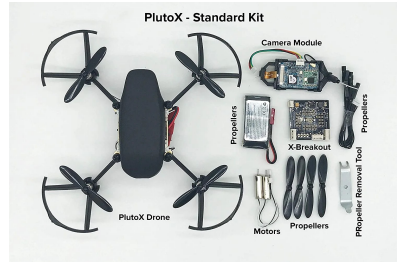
shortlist

Round 2: Hardware

top teams from Round 1
the same code flies the **real**
Pluto X in a real arena

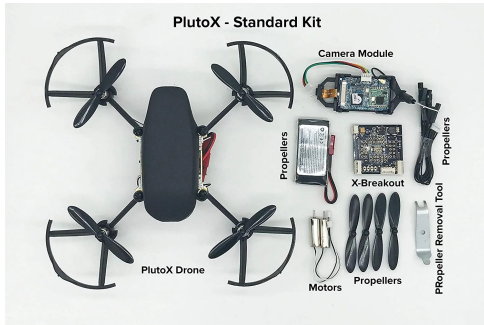


The simulated arena (Gazebo)



The real Pluto X kit

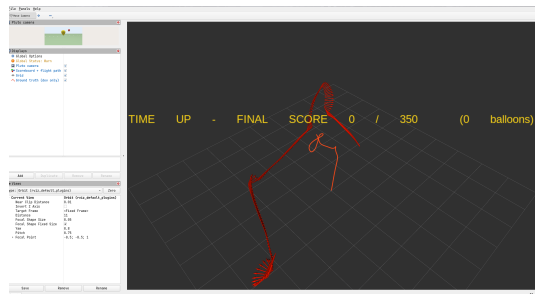
The hardware: Pluto X



Maker	Drona Aviation
Size, weight	16 × 16 × 4 cm, 68 g with the camera
Motors	4 brushed coreless, 55 mm propellers
Battery	1S LiPo 3.7 V, 600 mAh (about 9 min)
Flight controller	STM32F303 running MagisV2 firmware
Sensors	ICM-20948 (gyro, accel, magnetometer), ICP-10111 barometer
Camera	1280 × 720, about 18 fps over Wi-Fi, $\approx 80^\circ$ field of view
Link	Wi-Fi to your laptop (MSP protocol)
You read	tilt, heading, barometric height, battery, armed
You send	roll, pitch, yaw rate, throttle

The simulation stack

- **Gazebo Harmonic**: physics and the 3-D world
- **ROS 2 Humble**: plumbing between the parts (hidden from you)
- The drone's **real MagisV2 firmware**, running unmodified in the loop
- Simulated **sensors** with realistic noise: gyroscope, accelerometer, magnetometer, barometer, battery, camera
- **Wind gusts**, command delay, motor lag, air drag, battery sag
- Scoring, recording of every run, analysis and evaluation tools



RViz: the drone's camera, its path, the score

Rounds, deadlines and expectations

Round 1: Simulation

Opens: **Sep 27** Deadline: **Sep 30, 11:59 PM**
Submit **all** of (in output/; see DELIVERABLES.md):

- 1 your controller: `outerloop_controller/`
- 2 your best run + its **analysis notebook**, and an **evaluation** (≥ 5 runs) + its notebook
- 3 a **video** of your best run: pops, red balloons avoided
- 4 a **technical report** (PDF, ≤ 8 pages)

`python3 output/collect.py` gathers 1–2.
Judged on layouts **you have not seen**.

Round 2: Hardware

Shortlist: **Oct 1** Final: **Oct 3**, drone arena

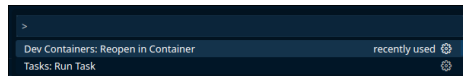
- The top teams of Round 1 are invited.
- The **same program** flies the real Pluto X (-hardware).
- Real video delay, real lighting, a real battery.
- Time to test and tune on site before the scored flights.
- The judges count the balloons.

Setup in three steps

- 1 Clone the repository
- 2 Choose how to run it:
 - **Docker dev container** (recommended; Linux, Windows via WSL2, macOS): VS Code → *Dev Containers: Reopen in Container*
 - **Local** (Ubuntu 22.04 only): install ROS 2 Humble and Gazebo Harmonic, then `colcon build`
- 3 Fly the example

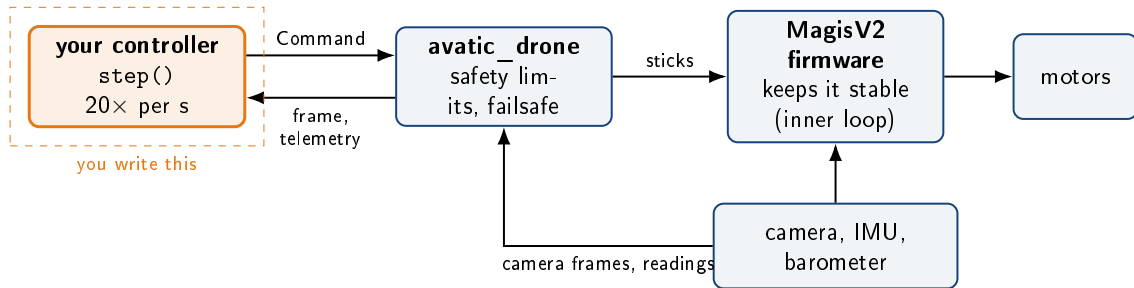
```
git clone -recursive https://github.com/Srindot/AVATIC.git
cd AVATIC
# (local setup only: source /opt/ros/humble/setup.bash
#                               && colcon build
#                               && source install/setup.bash)
ros2 launch pluto_x_bringup competition.launch.py \
  controller:=outerloop_controller/examples/hello_drone.py
```

Step-by-step instructions for every operating system: `workshop/manual.md` in the repository.



VS Code: open the dev container

How it fits together: two control loops



The **inner loop** (firmware) holds the tilt and turn rate you ask for. The **outer loop** (you) decides where to go, from the camera.

What you get, and what you write

Given to you

- `my_controller.py`: template that takes off and hovers
- `avatic_drone`: the drone interface (sim and real)
- `hello_drone.py`: example using the camera and commands
- **demo**: a pre-planned route popping every good balloon (fixed layout)
- **analysis**: every run recorded + a notebook
- **evaluation**: test on many hidden-style layouts
- the participant guide (`docs/participants/`)

You write

`MyController.step()` in
`outerloop_controller/my_controller.py`

```
def step(self, frame, telemetry, t):  
    # frame:      camera picture  
    # telemetry: tilt, heading,  
    #           height, battery  
    # t:         seconds since arming  
    ...  
    return Command(roll=0.0,  
                   pitch=0.0,  
                   yaw_rate=0.0,  
                   throttle=0.76)
```

Plain Python (NumPy, OpenCV). No ROS needed.

Commands and readings

You send (Command)

	range	meaning
roll	-1...1	tilt right (+): slides right; $0.2 \approx 7^\circ$, max 20°
pitch	-1...1	nose down (+): flies forward
yaw_rate	-1...1	turn clockwise (+), $\approx 77^\circ/\text{s}$ per unit
throttle	0...1	lift; ≈ 0.76 hovers

Safety limits (sim and real)

tilt ≤ 0.6 , yaw rate ≤ 0.8 , throttle ≤ 0.95 ; above 2.5 m it comes down; no command for 0.5 s $\Rightarrow$ it levels itself.

You read (telemetry)

altitude_m	height, from the barometer
heading_deg	0 = north, 90 = east (start), clockwise
roll_deg	+ = right side down
pitch_deg	+ = nose up
battery_v	battery voltage
armed	motors running

No position, no speed

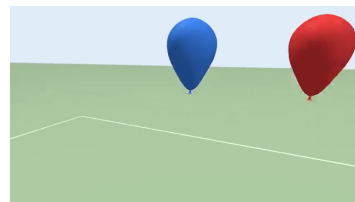
like the real drone: the camera is your main sensor.

The camera

- 1280×720 RGB, about 18 frames/s, $80^\circ \times 50^\circ$ view
- under the nose, looking ahead: **it tilts with the drone**
- mount: 3.5 cm forward, 1.8 cm below the centre
- pinhole model, no distortion:
$$K = \begin{bmatrix} 762.7 & 0 & 640 \\ 0 & 762.7 & 360 \\ 0 & 0 & 1 \end{bmatrix}$$
- direction to a pixel u :
bearing = $\arctan((u - 640)/762.7)$
- distance to a balloon (30 cm wide):
 $d \approx 229 / \text{width in pixels metres}$



searching (turning on the spot)



a target in view (blue), a red one to avoid

Running the simulator

```
ros2 launch pluto_x_bringup competition.launch.py controller:=outerloop_controller/my_controller.py
```

Option (name:=value)	what it does	default
controller:=<file>	the controller to run (any path)	none
headless:=true	no Gazebo window: faster	false
rviz:=false	no RViz window	true
arena_seed:=random / :=7	a random / a specific balloon layout	development layout
time_limit_s:=40	longer runs while developing (then "not official"; judging: 25)	25
record:=false	do not save this run	true
record_dir:=<folder>	save runs elsewhere	analysis/runs

Other commands

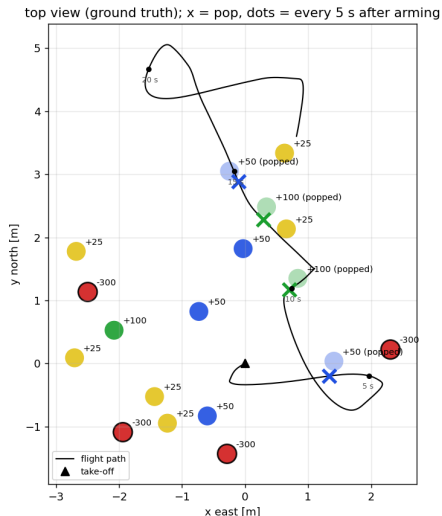
- demo: `ros2 launch pluto_x_demo balloon_demo.launch.py`
- new practice layout: `python3 analysis/new_seed.py`

Good to know

- one run per launch: Ctrl-C, then start again
- every run is saved automatically

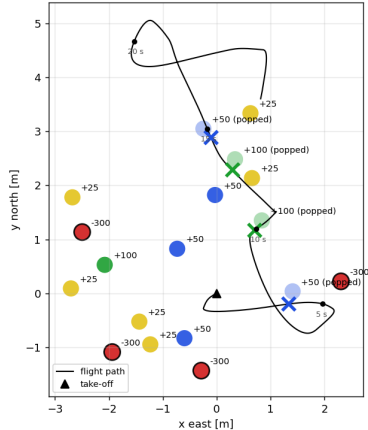
Analysis: understand one flight

- Every run is saved in `analysis/runs/`
- Open `analysis/analysis.ipynb`, run all cells
- Your **score** first, then:
 - a map of the flight over the balloons
 - height, distance to every balloon
 - your commands over time
 - camera pictures at key moments
- The map uses the **true** position: your controller never gets it, but you can learn from it

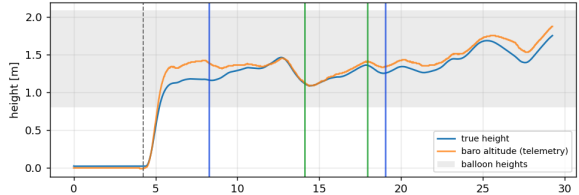


Analysis: what happened, over time

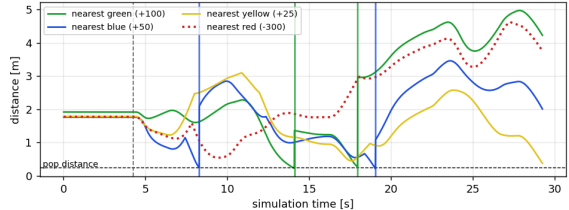
top view (ground truth); x = pop, dots = every 5 s after arming



altitude (dashed = armed, coloured lines = pops)



distance to the nearest balloon of each colour (dotted = red, a penalty)

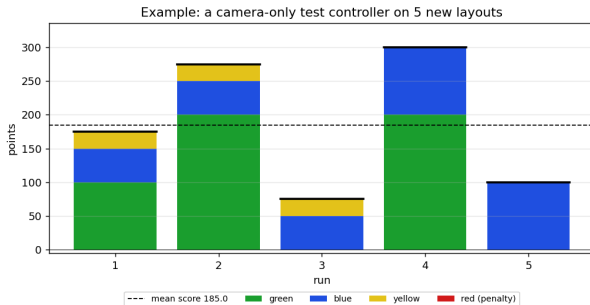


Top view, height, and the distance to the nearest balloon of each colour (dotted: red). Did you get close to the one you wanted?

Evaluation: test like the judges

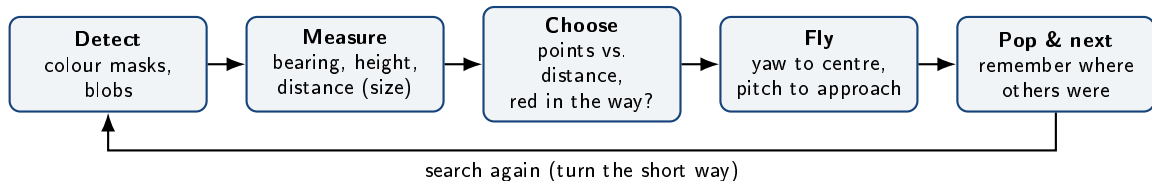
```
python3 evaluation/evaluate.py \  
-controller \  
  outerloop_controller/my_controller.py \  
-runs 5
```

- 5 runs on 5 **random layouts**, no windows (about 40 s each)
- then evaluation/evaluation.ipynb: **average score**, worst and best run, red hits, balloons per colour
- a controller that never arms scores 0
- runs vary: judge a change over several runs



A camera-only test controller: scores vary a lot between layouts

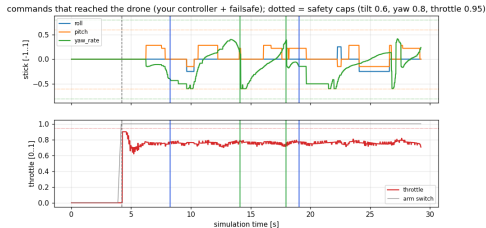
Solution ideas: a pipeline, not a solution



- Structure it as a **state machine**: take off → search → approach → pop → ...
- Work on a **smaller image** (`image[:4, :4]`) for speed
- A **proportional** yaw on the pixel offset is a good start
- Go for **green** first: 100 vs. 25 points
- Check the path for **red** before pushing forward
- Use the heading to **remember** balloons you have seen

Solution ideas: the hard parts

- **Drift.** There is no speed reading. If a balloon keeps sliding sideways in the picture, you are drifting: correct with roll.
- **Momentum.** The drone keeps its speed: brake before you arrive.
- **Time.** A full turn takes ≈ 6 s of your 25 s. Plan the search.
- **The camera tilts** with the drone: pitching forward moves the scene up in the picture.
- **Throttle:** change it smoothly, never cut it in the air.
- **Real video is late** (0.15–0.4 s): smooth beats fast.



Commands of the 300-point run

Solution ideas: go further, anything goes

Allowed, and encouraged

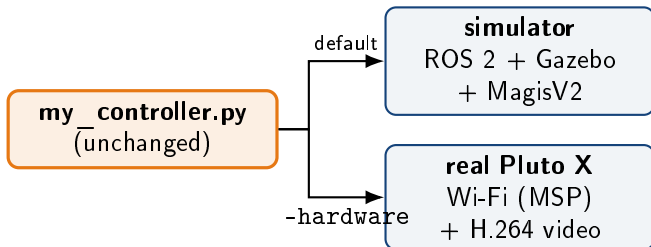
- **Learning-based models:** train a balloon detector (a small CNN or YOLO on simulator images), learn a control policy, try reinforcement learning in the simulator
- **Monocular mapping and memory:** estimate each balloon's position from its size and bearing in the image, keep a map of what you saw while scanning
- **Planning:** choose the order (green first, around the red): a small travelling-salesman problem
- **Visual odometry:** estimate your own motion from the camera (there is no position reading)
- Any Python library (NumPy, OpenCV, PyTorch, ONNX, ...; extra ones in `requirements.txt`), and your own files (model weights, parameters)

What still applies

- Only the **camera** and the **flight controller's readings**: no simulator ground truth (fair-play checks)
- Runs in **real time** on a laptop (20 decisions/s)
- Must work on the **real drone**: real lighting, video delay, a different hover throttle; do not over-fit to the simulator's clean images
- AI tools are allowed: **declare** them in your report

Ingenuity counts for 25 % of the shortlist: an interesting idea that scores a little less can still go through.

Sim2real: the same program on the real Pluto X



```
python3 outerloop_controller/my_controller.py -hardware
```

The same on both

- the same commands, readings and sign conventions
- the same firmware and safety limits

Different on the real drone

- video delay, fewer frames per second
- real lighting: re-tune your colour ranges
- hover throttle depends on the drone and battery
- landing at the end, Ctrl-C = land, twice = motors off (the simulator just pauses)
- no scoreboard: the judges count

Sim2real: how to prepare

- 1 **Be robust in simulation:** good average over many random layouts, not one lucky run
- 2 **Don't rely on exact numbers:** colours, sizes and timings will change
- 3 **Design for delay:** slow, smooth commands; don't react to a single frame
- 4 **Make thresholds easy to change:** keep them at the top of your file
- 5 **On site:** collect a few real camera frames first, check colours and hover throttle, then fly
- 6 **Safety first:** Ctrl-C to land, keep people clear



How teams are shortlisted for the final

Criterion	Weight	How it is measured
Simulation score	50 %	the organisers' own evaluation on hidden layouts (mean score), not your own numbers
Ingenuity, technical depth	25 %	your approach (perception, planning, control): more than the baseline?
Real-hardware readiness	15 %	real time, no hard-coded hover throttle, copes with video delay and lighting, no simulator-only tricks
Analysis and report	10 %	failure cases, what did not work, honest results

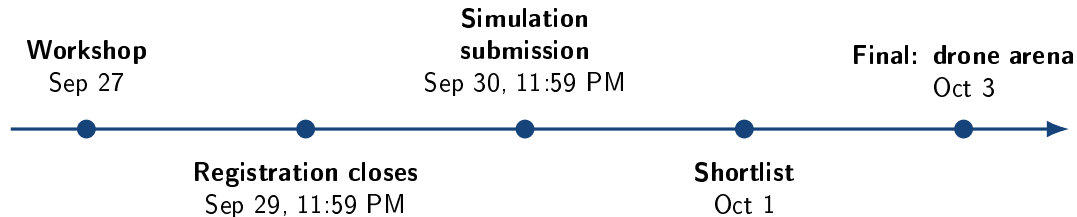
Gate: required to be considered

All deliverables submitted; the fair-play checks clean (or every finding explained).

Tie-breakers

Fewer red balloons hit, then more consistent scores across layouts.

Deadlines



Submission (Round 1)

controller code + executed analysis notebook (best run) + executed evaluation notebook + video + report



Prizes worth ₹ 20,000

Team size: 2–4 members. Register by **Sep 29, 11:59 PM:**

<https://forms.gle/6x25JHhfRKqESipF8>

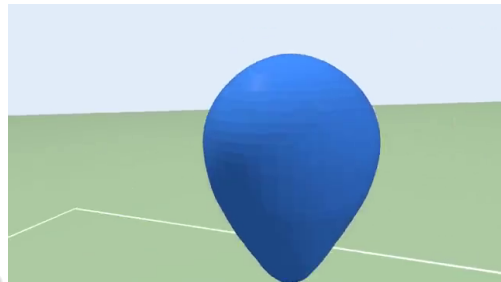
More Infinium events: <https://felicity.iiit.ac.in/infinium/events>

Get started today

- 1 Clone github.com/Srindot/AVATIC
- 2 Follow `workshop/manual.md` for your system
- 3 Fly `hello_drone.py`, then open `my_controller.py`
- 4 Read the participant guide:
`docs/participants/`

Questions?

✉ Srinath Bhamidipati
srinath.bhamidipati@research.iiit.ac.in



Happy popping!